KernelMind Project Report

SoC 2026 | Operating Systems, Reinforcement Learning, and CPU Scheduling

Dilip Kumar | Dual Degree, Electrical Engineering, IIT Bombay

Submission-focused report covering assigned work completed, implementation details, results, and analysis.

1. Assigned Work Completion Summary

The KernelMind project was completed across four assigned stages. The work progresses from operating-system scheduling theory to tabular reinforcement learning, then to RL-based scheduling-policy selection, and finally to direct neural scheduling. The implementation produces runnable outputs, saved metrics, plots, and evaluated results for the practical stages.

Assigned work	Completion status	Main completed output
Assignment 1 - Process abstractions and scheduling foundations	Completed	Solved analytical questions on process states, fork/wait behavior, interrupts, scheduling algorithms, wait time, turnaround time, makespan, and fairness.
Assignment 2 - The Adrian Descent	Completed	Implemented a tabular Q-learning landing controller with stochastic wind, reward shaping, saved Q-table, training metrics, learning curve, and greedy-policy evaluation.
Assignment 3 - Hybrid Meta-Scheduler	Completed	Implemented an RL meta-agent that selects among FCFS, SJF, and Round Robin; trained for 20,000 episodes and evaluated against static baselines.
Assignment 4 - Direct Neural Scheduler	Completed	Implemented a Direct-DQN scheduler that chooses ready-queue slots directly using masking, replay, target network updates, and attention-based Q-value prediction.

Completion here means that the assigned objective was implemented and evaluated. For Assignment 4, completion does not imply that the neural scheduler beats every classical baseline; the result is analyzed honestly because SJF remains stronger on the main benchmark.

2. Assignment 1 - Process Abstractions and Scheduling Foundations

Assignment 1 establishes the operating-system background needed for the later scheduling agents. The completed work explains process states, context switching, blocking behavior, zombie reaping, fork semantics, classical scheduling algorithms, and scheduling metrics.

2.1 Work completed

- Explained the timer-interrupt path: user execution is interrupted, hardware saves context, the kernel accounts for CPU time, the scheduler updates PCB state, and a runnable process is restored.
- Explained the correct blocking-read transition as Running -> Blocked -> Ready -> Running, and ruled out invalid transitions such as Terminated -> Blocked.
- Analyzed zombie process behavior: a process remains as a zombie until the parent collects its exit status using wait()/waitpid(), or until a subreaper/init-like process takes responsibility.
- Solved the nested fork behavior by using parent-child waiting and private address-space reasoning after fork.
- Compared FCFS, SJF/SRTF, and Round Robin using average waiting time, average turnaround time, response fairness, and Jain fairness.

Computed/argued item	Result or interpretation

Nested fork print ordering	C before B before A, because B waits for C and A waits for B.
Round Robin with q=3 average wait	12.2 ms for the given 5-process workload.
Round Robin with q=3 average turnaround	18.2 ms for the same workload.
Non-preemptive SJF average wait	8.0 ms for the same workload.
Non-preemptive SJF average turnaround	14.0 ms for the same workload.
FCFS Jain fairness example	0.746.
SJF Jain fairness example	0.547, showing lower equality of wait distribution despite better mean wait.

2.2 Analysis

This assignment is complete because it covers both conceptual process mechanics and quantitative scheduler comparisons. The key learning is that better average waiting time and better fairness are not the same objective. This distinction directly motivates the reward-design choices in Assignments 3 and 4.

3. Assignment 2 - The Adrian Descent

Assignment 2 converts a controlled-landing problem into a Markov decision process and solves it using tabular Q-learning. The completed agent learns when to enable thrust so that the probe lands safely under stochastic wind disturbances.

3.1 Environment and learning design

- State representation: altitude, velocity, and wind index. Altitude and velocity are discretized for tabular learning, while wind is represented as one of three Markov states: Calm, Gusty, and Adrian Gale.
- Action space: OFF or full thrust in the main version; the optional extension adds a variable-thrust action set with OFF, 50%, and 100% thrust.
- Dynamics: gravity, thrust, and drag are combined into a net force; velocity is updated using Euler integration, followed by altitude update.
- Terminal outcomes: safe landing, crash, runaway altitude, or timeout.
- Q-learning: the agent uses a 50 x 50 x 3 x 2 Q-table, epsilon-greedy exploration, Bellman updates, saved Q-table export, and greedy-policy evaluation.

Metric	Observed result
Training episodes	15,000
Final epsilon	0.020
Q-table size	50 x 50 x 3 x 2 = 15,000 Q-values
Last 1,000 episode success rate	98.5%
Last 500 episode success rate	98.6%
Last 100 episode success rate	99.0%
Last 1,000 average reward	1043.98
Last 1,000 mean impact velocity	-1.971 m/s
Greedy evaluation run	Success in 522 steps, impact velocity -2.064 m/s, total reward 1106.55

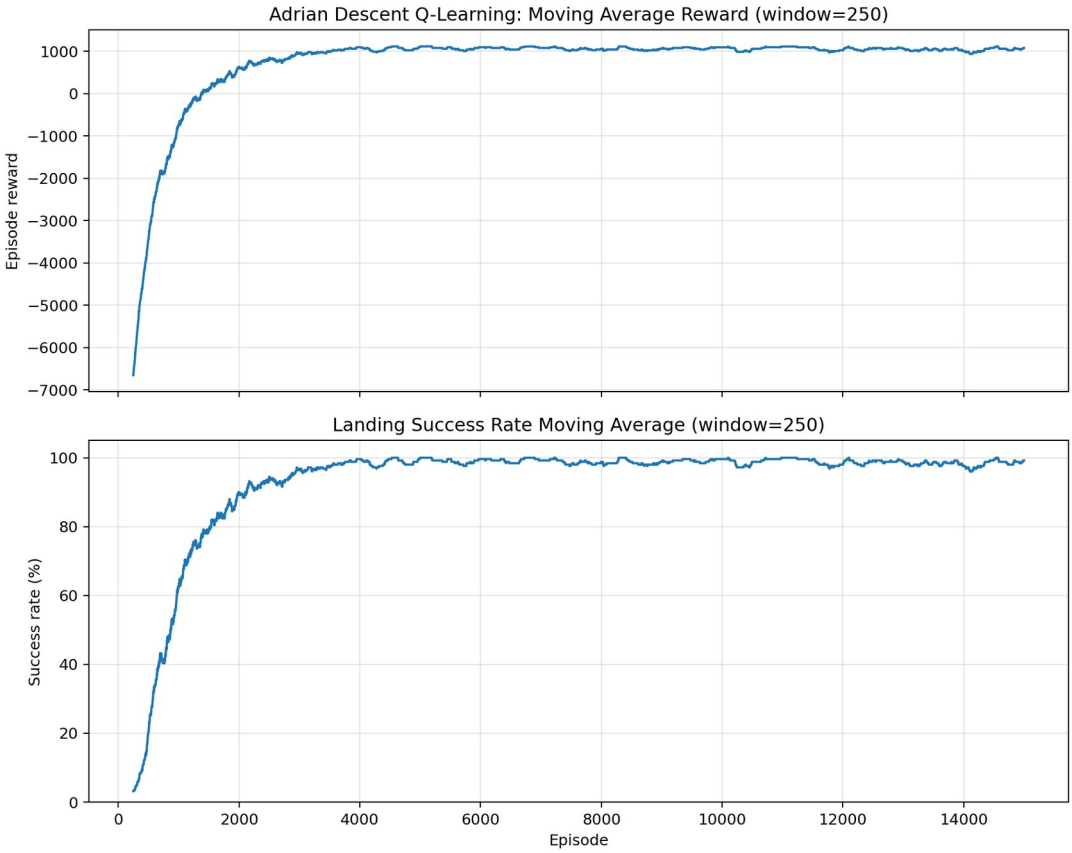

Figure 1. Assignment 2 learning curve showing reward improvement and landing success rate.

3.2 Results and analysis

The final policy is strong for a tabular method. The last 1,000 training episodes reach 98.5% success, and the greedy-policy run lands successfully with impact velocity safely above the -3.0 m/s crash threshold. The remaining failures are expected because discretized states and stochastic wind can still push the probe into rare unsafe trajectories. The result demonstrates that the reward design and state discretization are sufficient for reliable control.

4. Assignment 3 - Hybrid Meta-Scheduler

Assignment 3 applies reinforcement learning to CPU scheduling by training an agent to choose among classical scheduling policies at each decision step. This reduces the action space while still allowing adaptive behavior across workload types.

4.1 Scheduler design

- Process model tracks arrival time, burst time, remaining time, start time, finish time, executed ticks, waiting time, and turnaround time.
- Workloads include uniform, convoy, I/O-storm-like, and mixed patterns so that no single baseline is always ideal.
- Actions correspond to scheduler choices: FCFS, SJF/remaining-time selection, and Round Robin.
- State is discretized using queue length, average remaining-burst bucket, and maximum current-wait bucket.
- The reward penalizes queue delay, excessive maximum wait, context switching, mean wait, P90 wait, and unfairness.

Policy	Mean wait	P90 wait	Jain fairness	Makespan
RL Meta-Scheduler	25.5176	50.7954	0.6384	100.188
RR(q=4)	37.6790	54.6386	0.8127	100.188
SJF	40.2556	66.4632	0.6968	100.188
Random Agent	41.8442	64.1144	0.7656	100.188
FCFS	49.8962	73.3022	0.7956	100.188

Action used by trained policy	Evaluation count	Fraction
FCFS	8403	16.91%
SJF	19740	39.72%
RR	21560	43.38%

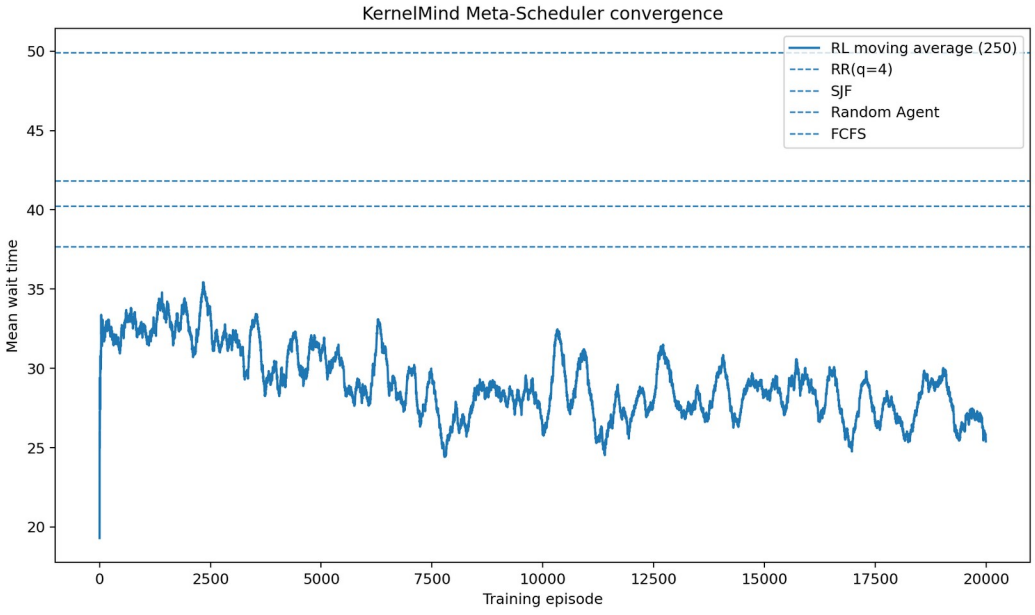

Figure 2. Assignment 3 convergence plot with static baseline reference lines.

4.2 Results and analysis

Assignment 3 is the strongest empirical result in the project. The RL meta-scheduler obtains the lowest mean wait and P90 wait on the evaluated workload distribution: 25.52 mean wait versus 37.68 for RR, 40.26 for SJF, 41.84 for

Random, and 49.90 for FCFS. The action distribution also shows that the agent did not collapse into one static policy; it used all three actions, with SJF and RR dominating.

The tradeoff is fairness. RR achieves higher Jain fairness than the RL meta-scheduler. This is not a failure of implementation; it reflects the selected reward objective, which prioritizes mean and tail latency more strongly than equality of waiting-time distribution.

5. Assignment 4 - Direct Neural Scheduler

Assignment 4 removes the heuristic-action shortcut and asks the agent to directly select a ready process. This makes the task harder because the action space depends on the visible queue, invalid padded slots must be masked, and the agent must learn process-selection behavior from delayed scheduling rewards.

5.1 Neural scheduler design

- State tensor: up to 10 ready processes, each represented by normalized remaining time, wait time, priority, virtual runtime, and arrival time.
- Action: a queue-slot index, not a process ID. Invalid padded slots are masked so they cannot be selected.
- Network: a self-attention Q-network produces one Q-value per visible queue slot.
- Training: replay buffer, epsilon schedule, Double-DQN style target computation, target-network synchronization, Huber loss, and gradient clipping.
- Reward: combines tick penalty, queue penalty, completion bonus, virtual-runtime fairness penalty, and starvation penalty.

Policy	Mean wait	P90 wait	Jain fairness
FCFS	34.960	63.518	0.210
SJF	23.858	58.460	0.395
RR	48.452	71.262	0.454
Random	47.203	73.053	0.406
Direct-DQN	33.060	64.740	0.304

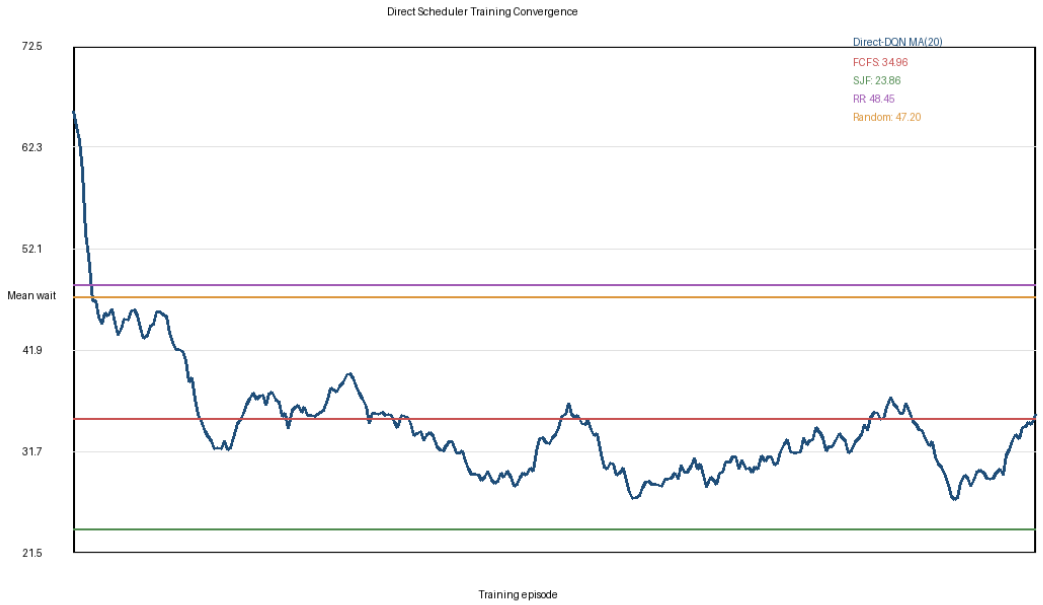

Figure 3. Assignment 4 Direct-DQN training convergence.

5.2 Optional objective results

Optional test	Best classical result	Direct-DQN / Agent result	Interpretation
Multicore scheduling	SJF mean wait 0.550, fairness 0.927	Agent mean wait 0.620, fairness 0.913	Close to SJF, but not best.
Memory-constrained scheduling	SJF mean wait 24.227, fairness 0.374	Agent mean wait 32.850, fairness 0.330	Better than RR/Random on mean wait, weaker than SJF.
I/O storm stress test	SJF mean wait 11.404, fairness 0.477	Direct-DQN mean wait 36.236, fairness 0.274	Weak out-of-distribution behavior.
Seed stability	Dominated by SJF fraction = 1.000	Mean wait 27.004 +/- 1.180, fairness 0.367 +/- 0.016	Stable, but still below the strongest baseline.

5.3 Results and analysis

The Direct-DQN implementation is complete and technically more advanced than the previous stages. It correctly handles variable ready queues, invalid-action masking, direct process selection, replay learning, and neural Q-value prediction.

The empirical result is mixed. Direct-DQN improves over RR and Random on mean wait in the core benchmark, but it does not beat SJF. It also has lower fairness than SJF, RR, and Random. The correct interpretation is that Assignment 4 demonstrates a working direct neural scheduler, but not a scheduler that is superior to simple deterministic baselines on the current synthetic workload set.

6. Overall Results and Analysis

Stage	Main result	Final assessment
Assignment 1	Process and scheduling theory solved with quantitative scheduler comparisons.	Complete foundation for the later RL scheduling work.
Assignment 2	98.5% success over the last 1,000 episodes; greedy run lands safely at -2.064 m/s.	Strong tabular RL result.
Assignment 3	RL meta-scheduler gives the best mean wait and P90 wait among evaluated policies.	Strongest empirical result in the project.
Assignment 4	Direct-DQN scheduler is implemented and evaluated, but SJF remains better on main mean-wait benchmark.	Technically complete, but not performance-dominant.

The project shows a clear learning and implementation progression. Assignment 2 demonstrates that tabular Q-learning can solve a constrained physical control problem when the state space is discretized effectively. Assignment 3 shows the best practical scheduling outcome by using RL to switch among strong heuristics. Assignment 4 shows the difficulty of replacing hand-designed heuristics with a direct neural policy: the neural agent is flexible, but the current training setup and workload distribution are not enough to beat SJF.

The final conclusion is that all assigned work is completed. The most successful results are the Adrian Descent controller and the Hybrid Meta-Scheduler. The Direct-DQN scheduler is a valid and complete extension, but its result should be presented as an experimental neural-scheduling implementation rather than as a proven improvement over classical scheduling.